

1. Why is lowercasing commonly applied during manual tokenization in NLP preprocessing? 1 point
- ☒ To reduce vocabulary size by treating "Word" and "word" as the same token
 - ☐ To improve model precision
 - ☐ To encode sentiment information such as positive, neutral and negative
 - ☐ To truncate long tokens so the lengths are the same size
2. When using nn.EmbeddingBag instead of nn.Embedding, what key advantage does it provide? 1 point
- ☒ It automatically pools embeddings across tokens
 - ☐ It eliminates the need for tokenization
 - ☐ It supports static but not dynamic embeddings
 - ☐ It ensures attention masks are applied
3. What is the main purpose of padding sequences in a batch? 1 point
- ☐ To convert words into one-hot vectors
 - ☒ To ensure all sequences in a batch are the same length
 - ☐ To truncate long sequences
 - ☐ To remove irrelevant tokens
4. When using pretrained embeddings, why would you "freeze" embedding weights? 1 point
- ☐ To reduce GPU memory usage
 - ☒ To prevent updates and retain pretrained representations
 - ☐ To improve tokenizer efficiency
 - ☐ To automatically generate attention masks
5. Which of the following is NOT a typical preprocessing step before sending input to a pretrained BERT model? 1 point
- ☐ Tokenizing into subwords or word pieces
 - ☐ Lowercasing text (for uncased models)
 - ☐ Padding sequences to a uniform length
 - ☒ Converting tokens into strings
6. Which method is commonly used to handle Out Of Vocabulary (OOV) words in natural language processing models? 1 point
- ☐ Dropping OOV words from the dataset to reduce complexity.
 - ☐ Using rule-based systems to manually define meanings for OOV words.
 - ☒ Replacing OOV words with a special token and utilizing word embeddings like subword tokenization or character-level modeling.
 - ☐ Ignoring the issue, assuming OOV words have minimal impact on model performance.
7. What happens when you apply a pretrained BERT tokenizer with padding=True, truncation=True, and return_tensors="pt" to a list of sentences? 1 point
- ☐ It converts text into character-level tensors
 - ☒ It outputs PyTorch tensors with padded/truncated input_ids and attention_mask
 - ☐ It trains a model automatically
 - ☐ It removes unknown tokens entirely
8. What is the purpose of this nn.Embedding layer in a text classifier? 1 point
- ```
self.embedding = nn.Embedding(vocab_size, embedding_dim)
```
- ☒ It encodes word indices into dense vectors that can capture semantic similarity
  - ☐ It directly predicts class probabilities from token indices
  - ☐ It applies padding and truncation automatically
  - ☐ It normalizes embeddings to unit vectors
9. What is Named Entity Recognition (NER) in the context of natural language processing? 1 point
- ☐ A technique for translating text from one language to another.
  - ☒ A process that identifies and classifies key elements from text into predefined categories such as names of people, organizations, locations, etc.
  - ☐ A method used to generate text summaries by extracting key phrases.
  - ☐ A system that evaluates the sentiment expressed in a piece of text, categorizing it as positive, negative, or neutral.
10. What key aspect should you consider when selecting pretrained embeddings? 1 point
- ☒ Vocabulary Size
  - ☐ Number of Epochs used in Training
  - ☐ Training Duration

Batch Size